

## Skill Week :- 2

### Task 1 :-

### Code:-

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 100

int main()
{
    char input[MAX_INPUT];

    printf("=====\n");
    printf("        SIMPLE INTERACTIVE SHELL\n");
    printf("=====\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        // Display prompt
        printf("MyShell> ");

        // Read user input
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        break;

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle exit condition
        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        // Handle empty input
        if (strlen(input) == 0)
        {
            continue;
        }

        // Display entered command
        printf("You entered: %s\n", input);
    }

    return 0;
}
```

### Output:-

```

anuan@Anushka:~$ nano ps.c
anuan@Anushka:~$ gcc ps.c -o ps
anuan@Anushka:~$ ./ps
=====
                SIMPLE INTERACTIVE SHELL
=====
Type 'exit' to quit.

MyShell> hello
You entered: hello
MyShell> pwd
You entered: pwd
MyShell> ls
You entered: ls
MyShell> exit
Exiting shell...
anuan@Anushka:~$ |

```

Task 2:-

Code:-

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("=====\n");
    printf("Enter Linux command (ls, pwd, date, whoami): ");
    scanf("%99s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0)
    {
        // Child Process
        printf("\n----- Child Process ----- \n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command: %s\n", command);

        execlp(command, command, (char *)NULL);

        // Executes only if exec() fails
        perror("Execution failed");
        exit(1);
    }

    else
    {
        // Parent waits for child
        wait(NULL);

        printf("\n----- Parent Process ----- \n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Child Process Finished Successfully.\n");
    }

    return 0;
}

```

Output:-

```
anuan@anushka:~$ gcc command_executor.c -o command_executor
anuan@anushka:~$ ./command_executor

=====
Enter Linux command (ls, pwd, date, whoami): pwd

----- Child Process -----
Child PID : 998
Parent PID : 997

Executing command: pwd

/home/anuan

----- Parent Process -----
Parent PID : 997
Child PID : 998

Child Process Finished Successfully.
anuan@anushka:~$
```